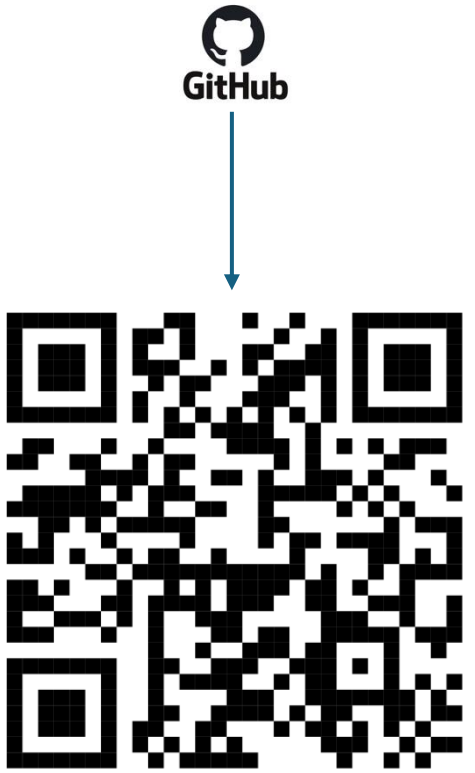
Snakemake Trainning 2025

Juan Leite



Introducing Snakemake Tool

Reproducible Data Analysis

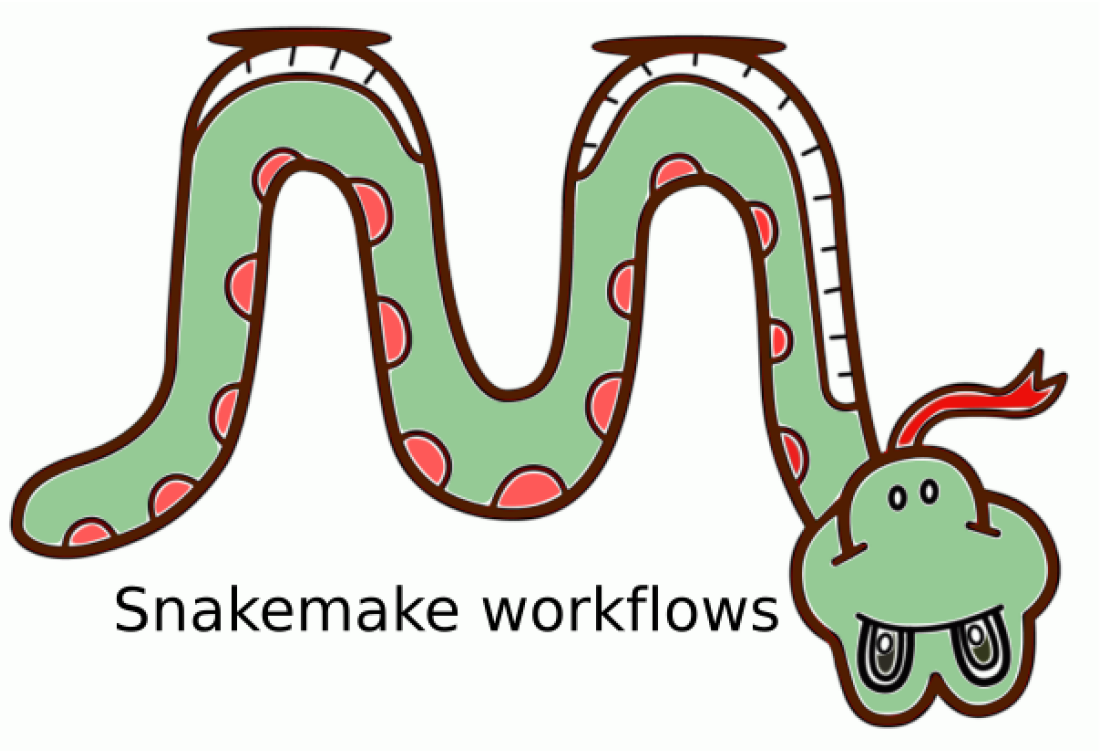
Snakemake ensures that data analysis processes are easy to reproduce and share, improving research collaboration and reliability.

Python-Based Rule Definition

Users define rules, steps, and dependencies with a straightforward Python-based language, making pipeline creation accessible and flexible.

Automation and Scalability

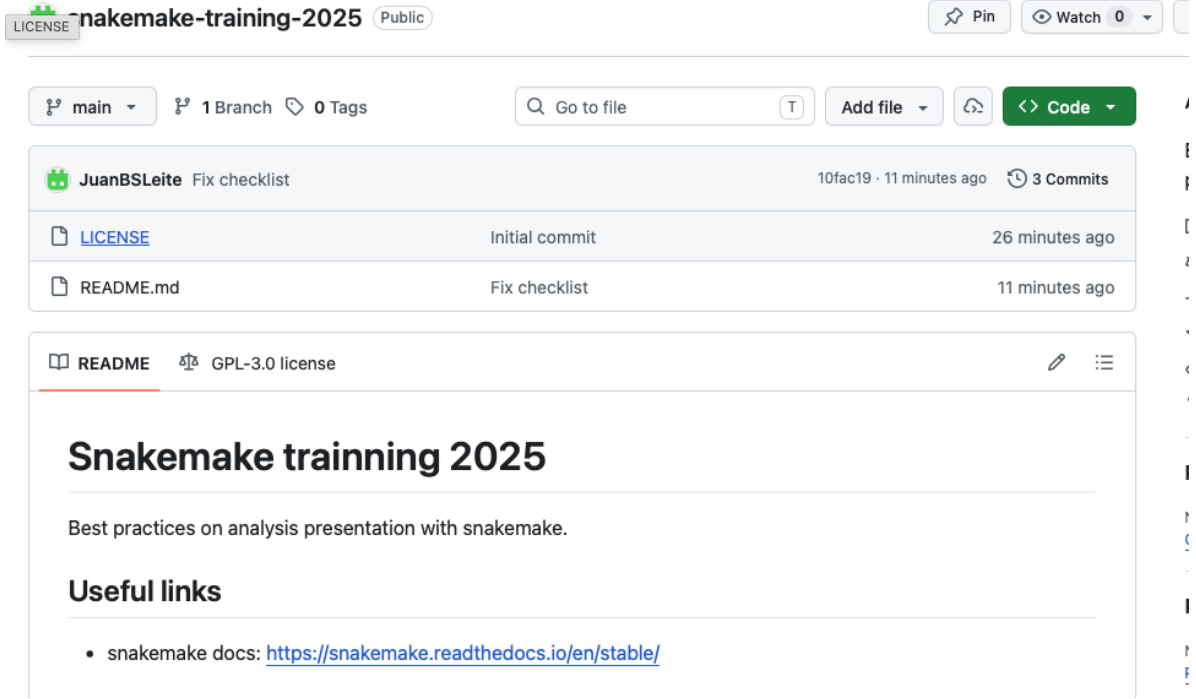
Snakemake automates complex analysis tasks and scales efficiently across different computing platforms and research environments.



Github

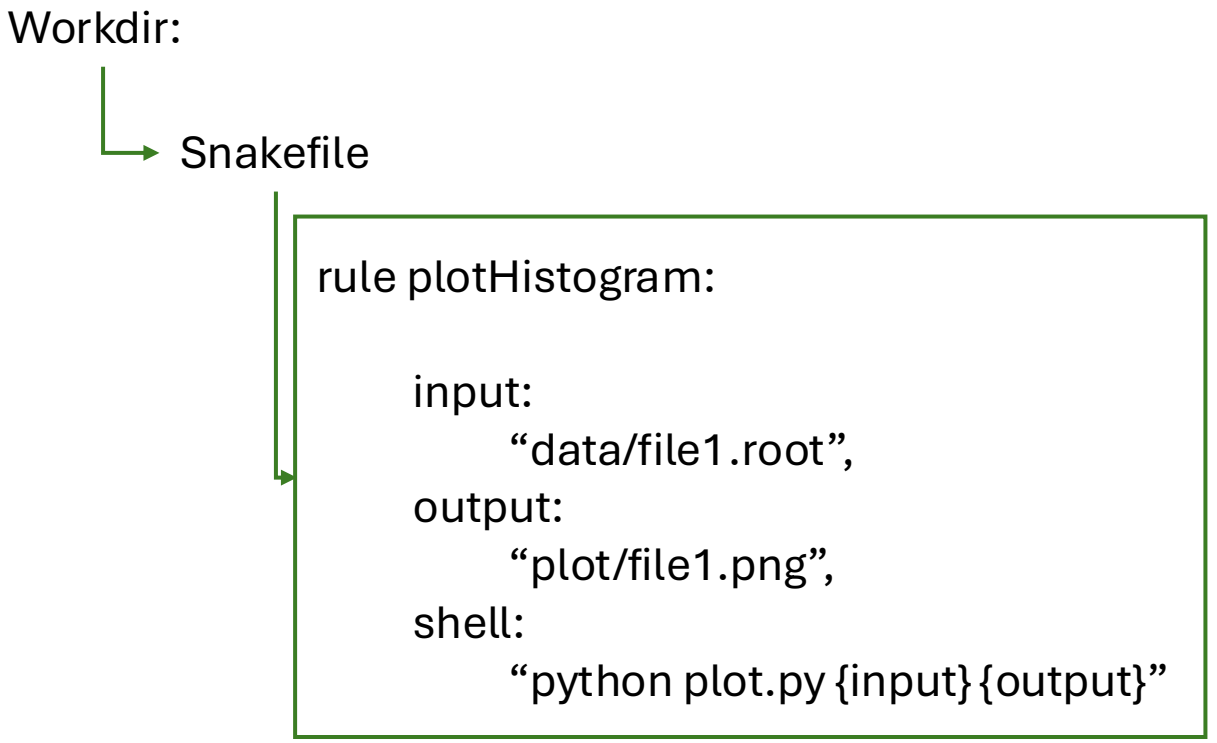
Useful links, Checklist, Slides, Notebooks and Exercises are available at the link below:

- Git hub: https://github.com/JuanBSLeite/snakemake-training-2025#



Basics: An example workflow

A Snakemake workflow is defined by specifying rules in a Snakefile. Rules decompose the workflow into small steps (for example, the application of a single tool) by specifying how to create sets of **output files** from sets of **input files**. Snakemake automatically **determines the dependencies** between the rules by matching file names.



$ snakemake -c1 plotHistogram

-c is the number of jobs

The **input** and **output** directives are followed by lists of files that are expected to be *used* or *created* by the rule.

The **shell** directive is followed by a string containing the shell command to execute.

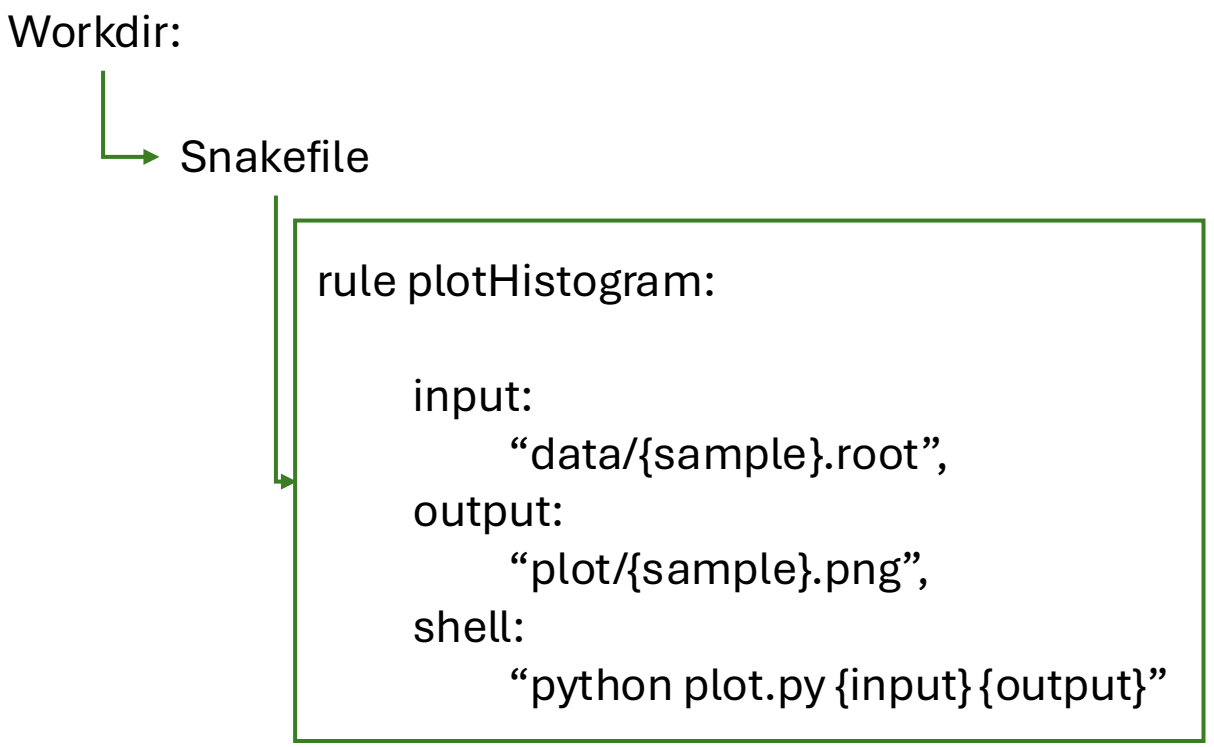
Generalizing rules

Snakemake allows **generalizing rules** through the use of **named wildcards.**

Let's replace file1 by the wildcard **sample** in the **input** and **output**.

Note:

You can have multiple wildcards in your output and input file paths. However, to avoid conflicts with other jobs of the same rule, **all output files** of a rule have to contain **exactly the same wildcards**.



$ snakemake -c1 plot/file1.png

When Snakemake determines that this rule can be applied to generate a target file, it will propagate that value to all occurrences of the {sample} wildcard in that rule, thereby determining the necessary input files for the resulting job.

Aggregating Inputs

Snakemake provides a **helper function for collecting input files** that helps us to describe aggregations.

- rule all is the top-level rule.
- expand() is a helper function that generates a list of filenames from a pattern.

Workdir:

Snakefile

```
samples = ["file1", "file2"]

rule all:
    input:
        expand("plot/{sample}.png", sample=samples)

rule plotHistogram:
    input:
        "data/{sample}.root"
    output:
        "plot/{sample}.png"
    shell:
        "python plot.py {input} {output}"
```

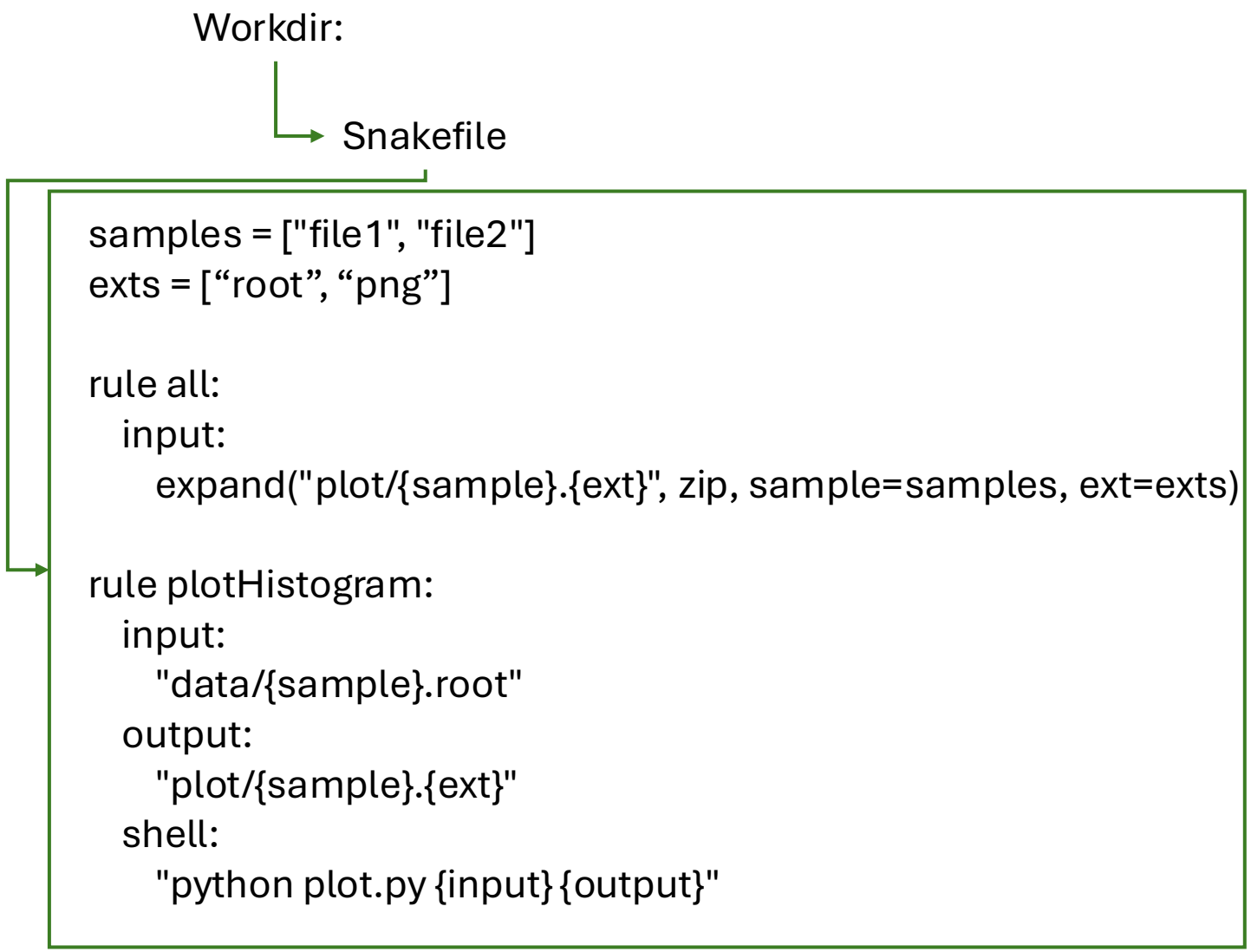
$ snakemake -c1 all

Aggregating Inputs II

It is also possible to expand multiple wildcards.

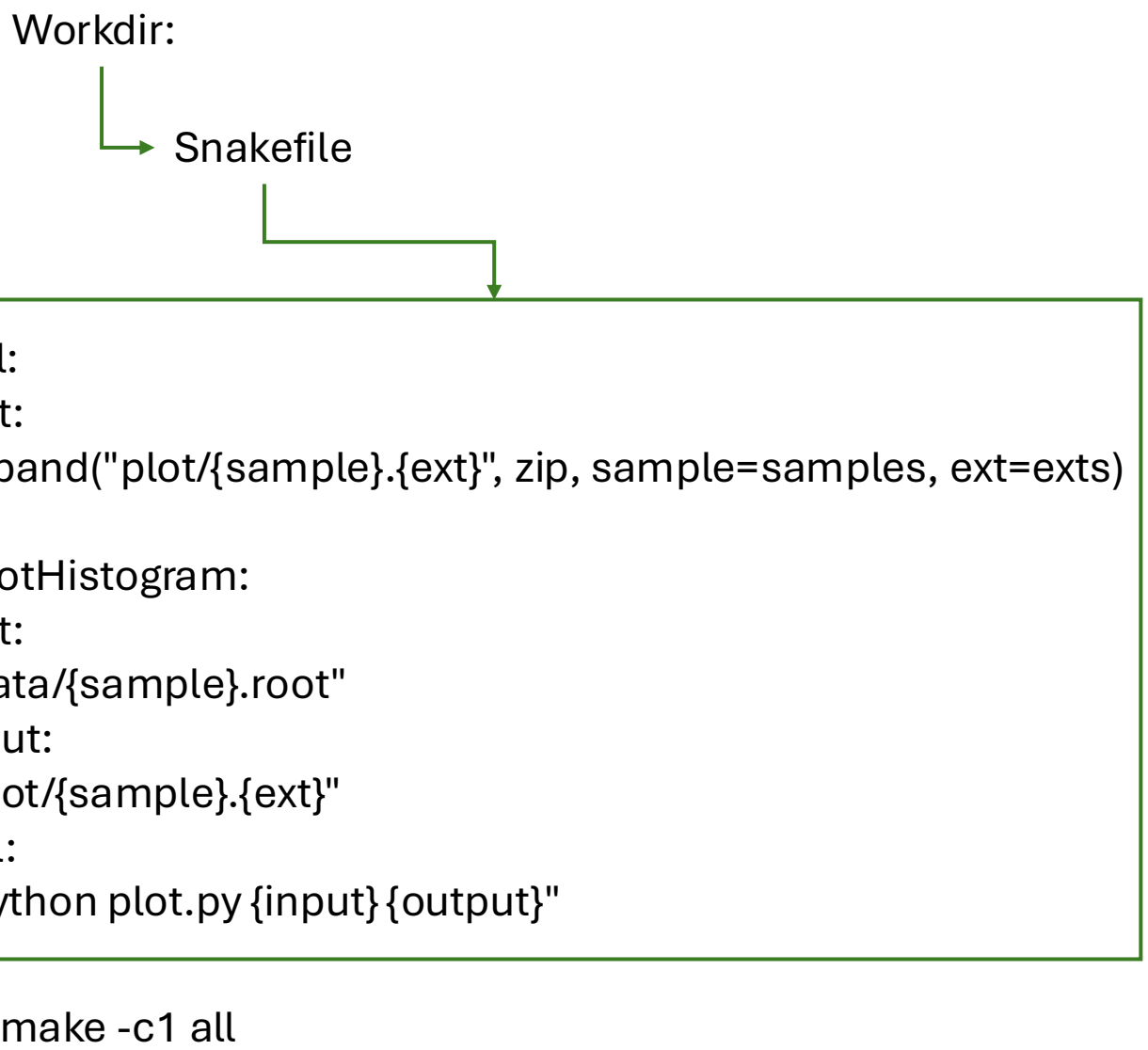
zip pair elements together position-by-position.

```
[
        “plot/file1.root”,
        “plot/ file1.png”,
        “plot/ file2.root”,
        “plot/ file2.png”,
]
```



$ snakemake -c1 all

Aggregating Inputs III



Using custom scripts

Snakemake offers the script directive. It is best practice to use the script directive whenever an inline code block would have more than a few lines of code.

Workdir:

→ Snakefile

```
samples = ["file1", "file2"]
exts = [“root”, “png”]
output = list()

for sample in samples:
   for ext in exts:
      output.expand(f“plot/{sample}.{ext}”)
```

```
rule all:
   input:
      expand("plot/{sample}.{ext}", zip, sample=samples,
ext=exts)

rule plotHistogram:
   input:
      "data/{sample}.root"
   output:
      "plot/{sample}.{ext}"
   script:
      "plot.py"
```

In the script, all properties of the rule like input, output, wildcards, etc. are available as attributes of a global snakemake object.

plot.py

```
file = read(“{snakemake.input[0]}/DecayTree”)
...
output = write(“{snakemake.output[0]}”)
```

$ snakemake -c1 all

Listing outputs

Rule outputs can be listed on shell by using the --summary option or –dry-run.

$ snakemake -c1 all --summary

or

$ snakemake -c1 all --dry-run

“plot/file1.root”
“plot/ file1.png”
“plot/ file2.root”
“plot/ file2.png”

Parallelizing tasks

Tasks can be parallelized by setting the number of workers (-c).

$ snakemake –c4 all

The previous command run all the 4 tasks and produce the outpus in parallel.

Resource management

Resource management in Snakemake is about controlling how much **CPU, memory, and time each rule is allowed to use**, so workflows run efficiently on laptops, clusters, or HPC systems.

Snakefile

```
rule plotHistogram:
    input:
        "data/{sample}.root"
    output:
        "plot/{sample}.png"
    threads: 2
    resources:
        mem_mb=2000
        runtime=10
        threads=4
    shell:
        "python plot.py {input} {output}"
```

What each field means?

- **threads**
  How many CPU cores the rule can use.
  Then you run Snakemake like

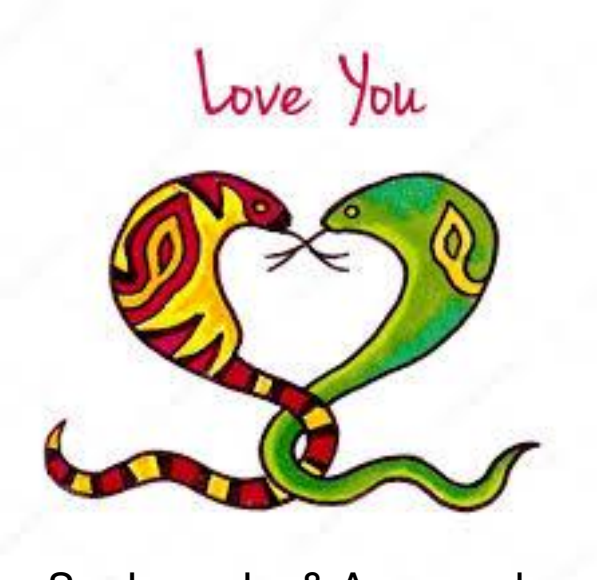
```
$snakemake -j 8
```

  Snakemake will schedule jobs so total threads ≤ 8.

- mem_mb
  maximum memory per job
- runtime
  maximum job time.

Environment setting with Conda

Using Conda with Snakemake is one of its most powerful features because it lets each rule run in its own isolated environment.



Snakemake & Anaconda

```
$ snakemake --use-conda -j 4
```

Snakefile

```
rule plotHistogram:
    input:
        "data/{sample}.root"
    output:
        "plot/{sample}.png"
    threads: 2
    conda:
            “envs/plot.yaml”
    shell:
        "python plot.py {input} {output}"
```

envs/plot.yaml

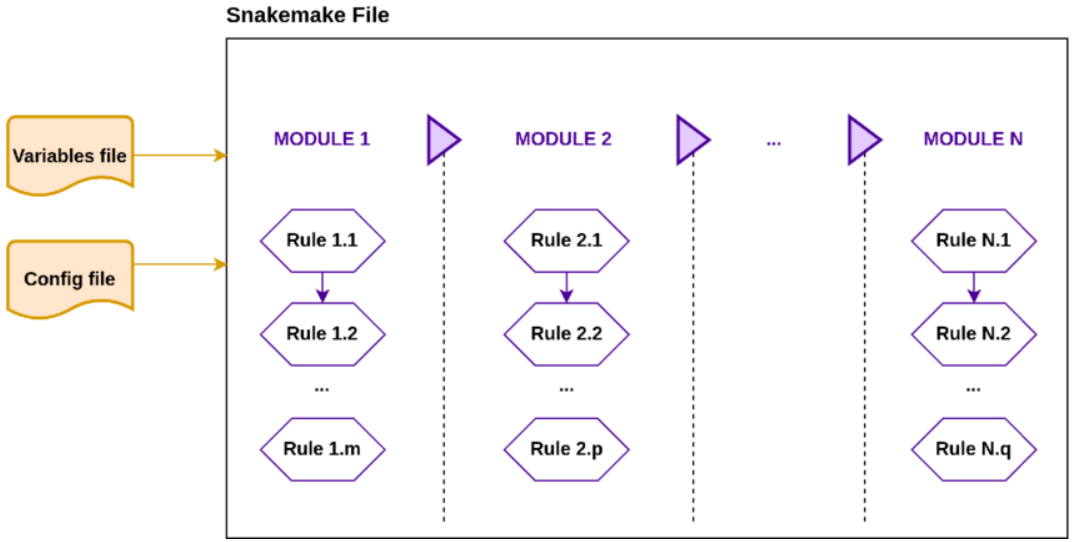
```
name: plot-env
channels:
  - conda-forge
  - defaults
dependencies:
  - python=3.10
  - matplotlib
  - numpy
  - uproot
```

This ensures:

- reproducible Python version
- correct packages
- no system conflicts

Modularization

Modularisation in Snakemake means splitting a large workflow into smaller reusable files (modules) instead of keeping everything in one huge snakefile.



Basic idea: Instead of one big snakefile, you split into:

```
workflow/
  Snakefile
  rules/
    plotting.smk
  envs/
  scripts/
```

Usage:

```
module mapping:
  snakefile:
    "rules/plot.smk"
```

Then use exported rules:

```
use rule * from plot
```

```
use rule plotHistogram from plot
```

Config files

In Snakemake, config files are used to separate parameters and settings from the workflow logic.

Instead of hardcoding values in your Snakefile (like sample names, file paths, thresholds), you put them in a YAML or JSON config file.

config.yaml

```
samples:
  - file1
  - file2

threshold: 0.05
outdir: plot/
```

Snakefile

```
configfile: "config.yaml"

rule all:
    input:
        expand("{outdir}{sample}.png",
               sample=config["samples"],
               outdir=config["outdir"])
```

Accessing config values

```
config["samples"]
config["outdir"]
```

Overriding config from command line

```
$snakemake --config outdir=new_plots
```

That’s enough to start with the snake way.

Thanks!